Image and Table Understanding in Retrieval-Augmented Generation (RAG)

1. Introduction

Retrieval-Augmented Generation (RAG) systems are designed to retrieve relevant information from a knowledge base and provide context-aware responses through a Large Language Model (LLM). Traditional RAG systems primarily focus on textual content and perform exceptionally well when documents contain only machine-readable text.

However, modern documents such as research papers, technical manuals, reports, scanned PDFs, presentations, and business documents often contain a combination of:

- Images
- Tables
- Charts
- Diagrams
- Flowcharts
- Infographics

Traditional RAG pipelines fail to effectively process these visual elements because they rely solely on text extraction. As a result, valuable information embedded within images and tables remains inaccessible to the retrieval system.

This document discusses the limitations of traditional RAG systems and presents a complete image and table processing pipeline for enhancing RAG capabilities.

---

2. Why Traditional RAG Fails for Images and Tables

2.1 Traditional RAG Pipeline

A conventional RAG pipeline generally follows the steps below:

PDF → Text Extraction → Chunking → Embedding Generation → Vector Database → Retrieval → LLM

This approach assumes that all useful information is available as text.

2.2 Problems with Images

Consider an architecture diagram:

[User] → [API Gateway] → [FastAPI] → [Pinecone] → [LLM]

A text extraction tool cannot understand:

- Components present in the image
- Relationships between components
- Direction of data flow
- Functional meaning of the diagram

As a result, the image is either ignored or stored as a meaningless object.

Example

Input Image:

Architecture diagram of a chatbot.

Traditional RAG Output:

No information extracted.

Expected Output:

"The user sends a request to the API Gateway. The API Gateway forwards the request to FastAPI, which retrieves relevant context from Pinecone before generating a response through the LLM."

2.3 Problems with Tables

Consider the following table:

Year	Revenue
2023	10 Cr
2024	15 Cr

Traditional OCR may extract:

2023 10 Cr 2024 15 Cr

The structural relationship between rows and columns is lost.

Expected Understanding:

"Revenue increased from 10 Cr in 2023 to 15 Cr in 2024, representing a 50% growth."

Without structure preservation, retrieval quality decreases significantly.

2.4 Consequences

Traditional RAG systems:

- Ignore visual information
- Lose table structure
- Fail to understand diagrams
- Cannot interpret charts
- Generate incomplete answers
- Reduce retrieval accuracy

Therefore, dedicated pipelines are required for images and tables.

---

3. Image Classification for RAG

3.1 Why Image Classification is Necessary

Not all images contain the same type of information.

A text-heavy image requires OCR, while a natural photograph requires image captioning.

Applying the same processing technique to all images results in poor performance.

Therefore, images must first be classified before processing.

3.2 Image Categories

Category 1: Text-Based Images

Examples:

- Scanned documents
- Book pages
- Notices
- Research paper pages

Tool

PaddleOCR

Pipeline

Image ↓ OCR ↓ Extracted Text ↓ Chunking ↓ Embedding ↓ Vector Database

Input Example

Image containing:

"Artificial Intelligence is transforming healthcare."

Output Example

Extracted Text:

"Artificial Intelligence is transforming healthcare."

Category 2: Images Containing Text and Figures

Examples:

- Technical documentation
- Research papers
- Engineering illustrations

Tools

- DocLayout-YOLO
- PaddleOCR
- BLIP-2

Pipeline

Image ↓ Layout Detection ↓ Text Region + Figure Region ↓ OCR + Caption Generation ↓ Merged Description ↓ Embedding

Input Example

A page containing a machine learning workflow diagram with explanatory text.

Output Example

"Data is collected and preprocessed before model training. The workflow includes feature engineering, model training, and evaluation stages."

Category 3: Natural Images

Examples:

- Animals
- Vehicles
- Buildings
- Medical Images

Tool

Florence-2

Pipeline

Image ↓ Image Captioning ↓ Text Description ↓ Embedding

Input Example

Image of a dog sitting near a lake.

Output Example

"A brown Labrador is sitting on grass near a lake."

Category 4: Charts and Graphs

Examples:

- Pie Charts
- Line Charts
- Bar Charts

Tools

- Florence-2
- LLaVA

Pipeline

Chart Image ↓ Chart Understanding ↓ Data Interpretation ↓ Summary Generation ↓ Embedding

Input Example

Monthly sales graph.

Output Example

"Sales increased steadily from January to June, reaching their highest value in June."

Category 5: Flowcharts and Architecture Diagrams

Examples:

- AWS Architecture
- UML Diagrams
- Network Diagrams
- System Architecture

Tools

- Florence-2
- PaddleOCR

Pipeline

Diagram ↓ Object Detection ↓ OCR ↓ Relationship Extraction ↓ Structured Summary ↓ Embedding

Input Example

RAG architecture diagram.

Output Example

"FastAPI retrieves context from Pinecone and sends it to Llama for response generation."

Category 6: Mixed Images

Examples:

- Research papers
- Technical reports
- Scientific documents

Tools

- Docling
- PaddleOCR
- Florence-2

Pipeline

Image ↓ Layout Detection ↓ Text Table Figure Formula ↓ Individual Processing ↓ Unified Context ↓ Embedding

---

4. Table Processing for RAG

4.1 Why Tables Need Special Handling

Tables contain structured relationships between rows and columns.

OCR alone extracts text but does not preserve structure.

Meaningful retrieval requires preserving these relationships.

4.2 Table Categories

Category 1: Simple Structured Tables

Example:

Name	Age
John	25

Tool

PaddleOCR Table Engine

Pipeline

Table Image ↓ OCR ↓ Markdown Conversion ↓ Embedding

Output

Name	Age
John	25

Category 2: Financial Tables

Example:

Year	Revenue
2023	10 Cr
2024	15 Cr

Tools

- PaddleOCR
- Table Transformer

Pipeline

Table ↓ Structure Extraction ↓ Semantic Summary ↓ Embedding

Output

"Revenue increased from 10 Cr in 2023 to 15 Cr in 2024."

Category 3: Research Paper Tables

Example:

Model	Accuracy
BERT	92%
RoBERTa	94%

Pipeline

Table ↓ Row Extraction ↓ Text Conversion ↓ Embedding

Output

"BERT achieved 92% accuracy while RoBERTa achieved 94%."

Category 4: Comparison Tables

Example:

Feature	Model A	Model B

Pipeline

Table ↓ Feature Comparison ↓ Summary Generation ↓ Embedding

Output

"Model B provides all capabilities of Model A along with additional support for Feature C."

Category 5: Large Multi-Page Tables

Examples:

- Bank Statements
- Audit Reports
- Inventory Records

Pipeline

Table ↓ Page-wise Processing ↓ Row Chunking ↓ Embedding

Chunk Example

Rows 1–50

Rows 51–100

Rows 101–150

Category 6: Scanned Tables

Tools

- OpenCV
- PaddleOCR

Pipeline

Image ↓ Noise Removal ↓ Deskewing ↓ OCR ↓ Table Reconstruction ↓ Embedding

---

Category 7: Complex Merged Cell Tables

Tools

- Table Transformer
- Docling

Pipeline

Table ↓ Cell Detection ↓ Header Hierarchy Recovery ↓ Structured JSON ↓ Summary ↓ Embedding

Output Example

{ "2024": { "Q1":100, "Q2":120 } }

---

5. Recommended Technology Stack

Image Processing

Image Classification:

- Florence-2

OCR:

- PaddleOCR

Layout Detection:

- DocLayout-YOLO

Image Captioning:

- Florence-2
- BLIP-2

Document Understanding:

- Docling

Table Processing

Table Detection:

- Table Transformer

Table OCR:

- PaddleOCR Table Engine

Image Enhancement:

- OpenCV

Complex Document Parsing:

- Docling

RAG Components

Embedding Model:

- all-MiniLM-L6-v2

Vector Database:

- Qdrant

Framework:

- LangChain

LLM:

- Llama 3.2

---

6. Final Proposed Pipeline

PDF / Image ↓ Document Segmentation ↓ Image Classification ↓ Category-Specific Processing

Text Images → PaddleOCR

Natural Images → Florence-2

Charts → Florence-2

Diagrams → Florence-2 + OCR

Tables → PaddleOCR + Table Transformer

↓ Rich Text Summary Generation ↓ Chunking ↓ Embedding Generation ↓ Qdrant Vector Database ↓ Retriever ↓ Llama 3.2 ↓ Final Response

---

7. Conclusion

Traditional RAG systems are designed primarily for text and fail to capture information stored in images and tables. To build a robust multimodal RAG system, images and tables must be classified and processed using specialized pipelines.

By integrating OCR, layout detection, image captioning, chart understanding, table extraction, and semantic summarization, visual content can be transformed into structured textual knowledge suitable for embedding and retrieval.

This approach significantly improves retrieval quality, context understanding, and answer accuracy in document-centric RAG systems.